# FLAG

FLAG · MÓDULO 03 · SESSÃO 2

# Branches e merge

Experimentar sem estragar: branches, merge e conflitos. O .gitignore e voltar atrás no histórico.

6.ª feira, 13 nov 2026 · 9:30–12:30 · MS Teams · Workshop: Versões de Código com GitHub (2 de 3)

# Boas práticas Live Training



Escolher um sítio calmo, sem distrações



Ser pontual



Desligar microfone. Ligar quando for para intervir



Manter a câmara ligada



Levantar a mão quando pretender participar



No intervalo, desligar a câmara e o microfone

HOJE

# Da 4.ª para hoje: o plano

1

### Branches

Uma linha paralela do projeto: experimentar sem estragar o main.

2

### Merge

Juntar o trabalho de um branch ao main. Os dois casos: avançar e juntar.

3

### Conflitos

Quando dois branches mudam a mesma linha. Resolver sem pânico.

4

### .gitignore e voltar atrás

O que o Git não deve ver. Corrigir e desfazer commits: amend, revert, reset.

**ANTES DE COMEÇAR: O LIVRO DA 4.ª**

```
$ cd .../livro  # a pasta onde fizeram o git init
$ git status
$ git log --oneline
```

### Na 4.ª

O ritmo status → add → commit, o diff e o restore. Hoje: tudo isto, mas em branches. Pausa às 10:50.

PARTE 1

# Branches

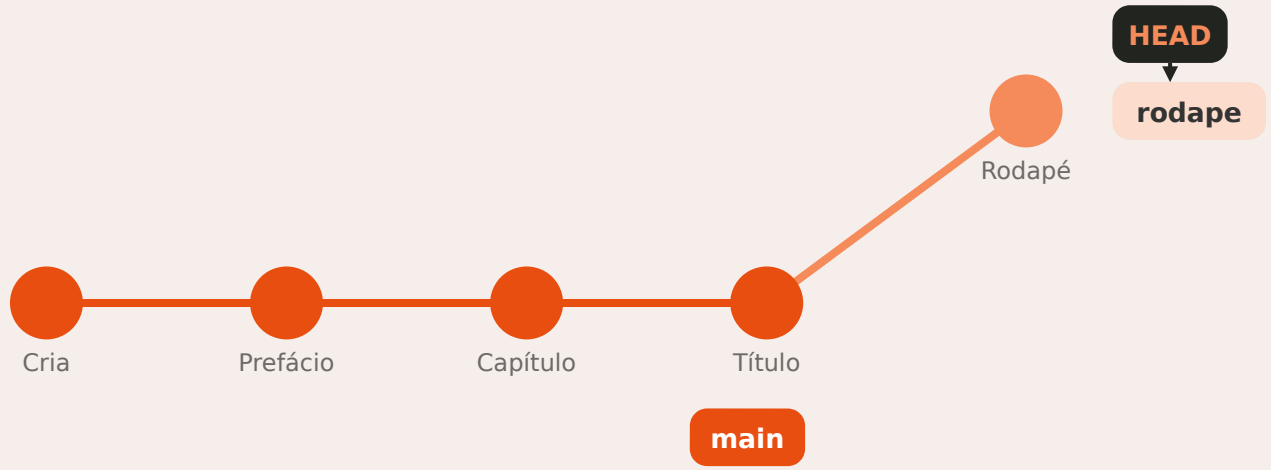
experimentar sem estragar o que já funciona

**O QUE É → OS COMANDOS → O RODAPÉ, JUNTOS**

TEORIA

# Um branch é uma linha paralela

### O livro, com um branch para o rodapé




**1 — Uma etiqueta num commit**

O branch é só um nome que aponta para um commit. Criar um é instantâneo.

**2 — Experimentar sem estragar**

O main fica como estava. Se a experiência correr mal, apaga-se o branch.

**3 — O HEAD: onde estou**

Aponta para o branch em que estou. Os commits novos vão para esse.

### Para que serve, no dia a dia

Uma funcionalidade nova, uma correção, uma ideia: cada uma no seu branch. Em equipa, cada pessoa no seu, e no fim junta-se tudo no main.

TEORIA

# Os comandos dos branches

1. **git branch**
   Lista os branches. O * marca onde estou.

2. **git switch -c rodape**
   Cria o branch rodape e muda para ele.

3. **git switch main**
   Muda de branch. Os ficheiros da pasta mudam também!

4. **git log --oneline --graph --all**
   O histórico de todos os branches, com o desenho.

5. **git branch -d rodape**
   Apaga um branch (depois de o juntar ao main).

NO LIVRO

```
$ git switch -c rodape
Switched to a new branch 'rodape'

$ git branch
  main
* rodape

# ... mudar o livro.txt, add, commit ...

$ git log --oneline --graph --all
* 9f1c2ab (HEAD -> rodape) Acrescenta o rodapé
* b4739da (main) Muda o título do livro
* 823da09 Acrescenta o capítulo 1
```

# O rodapé num branch

1. No livro: git status (limpo) e git branch (só o * main)
2. git switch -c rodape e git branch: o * mudou
3. No fim do livro.txt, o rodapé: © 2026 · o vosso nome · git add · git commit -m "Acrescenta o rodapé"
4. git log --oneline --graph --all: o rodape vai à frente do main
5. git switch main: o rodapé desaparece do livro.txt! · git switch rodape: volta
6. git switch main e o desenho outra vez: o main não sabe do rodapé

RESUMO

# O que escrevemos: o rodapé

LIVRO

```
$ git switch -c rodape
$ git add livro.txt
$ git commit -m "Acrescenta o rodapé"

$ git log --oneline --graph --all
* 9f1c2ab (HEAD -> rodape) Acrescenta o rodapé
* b4739da (main) Muda o título do livro

$ git switch main
$ git switch rodape
$ git switch main
```

## O branch é uma etiqueta

O rodape aponta para o commit novo; o main ficou onde estava.

## Mudar de branch muda a pasta

O Git troca os ficheiros pelos do commit desse branch. Por isso: commit antes do switch.

PARTE 2

# Merge

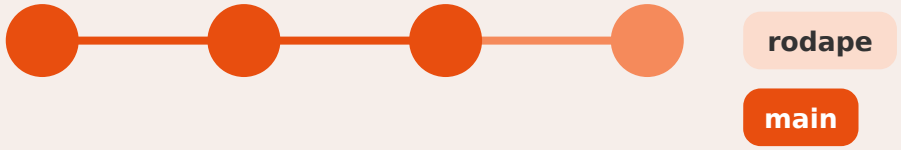
juntar o trabalho de um branch ao main

**OS DOIS CASOS → O RODAPÉ E O TÍTULO, JUNTOS → O EXERCÍCIO 4.1**

TEORIA

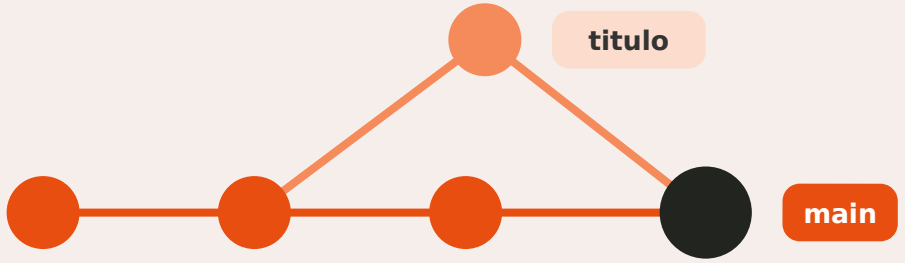
# Juntar um branch: os dois casos

## 1 · Fast-forward: o main só avança




O main não mudou desde que o branch saiu: a etiqueta só tem de avançar. Não há commit novo.

## 2 · Merge commit: os dois mudaram




O main também andou: o Git cria um commit de merge, com dois "pais". O desenho fica um losango.

NO MAIN, JUNTAR O BRANCH

```
$ git switch main  # ir para quem recebe
$ git merge rodape  # trazer o rodapé para aqui
```

# Juntar o rodapé e o título

1. git switch main · git merge rodape: fast-forward · git branch -d rodape
2. git switch -c titulo · mudar o título no livro.txt · add e commit -m "Muda o título"
3. git switch main · uma frase nova no capitulo1.txt · add e commit -m "Continua o capítulo 1"
4. git merge titulo: abre o VS Code com a mensagem do merge: gravar e fechar o separador
5. git log --oneline --graph --all: o losango · git branch -d titulo
6. No livro.txt: o título novo e o rodapé, os dois no main

RESUMO

# O que escrevemos: os dois merges

LIVRO

```
$ git switch main
$ git merge rodape
Fast-forward
$ git branch -d rodape

$ git switch -c titulo
$ git commit -am "Muda o título"
$ git switch main
$ git commit -am "Continua o capítulo 1"
$ git merge titulo
Merge made by the 'ort' strategy.

$ git log --oneline --graph --all
*   e41d0c3 (HEAD -> main) Merge branch 'titulo'
|\
| * 7a2f9d1 Muda o título
* | 5c8e1b4 Continua o capítulo 1
|/
```

## Quem recebe faz o merge

git switch main, e depois git merge <branch>.

## O atalho -am

git commit -am "…" faz o add e o commit de uma vez, mas só para ficheiros que o Git já conhece.

EXERCÍCIO

# Branches no site das receitas

1. No repositório das receitas (o do 3.1): `git status` limpo e `git log --oneline`
2. Branch `nova-receita`: acrescentar uma receita à lista do `index.html` e criar a página dela (2 commits)
3. Voltar ao `main`: a receita não está lá. Branch `cores`, a partir do main: mudar a cor do cabeçalho (1 commit)
4. No `main`: juntar a `nova-receita` (fast-forward) e depois as `cores` (merge commit)
5. Ver as duas mudanças no browser e apagar os dois branches
6. Colar o vosso `git log --oneline --graph` no chat

## Regras

Sem copiar comandos: os slides de resumo só se encravarem.

Sem o repositório das receitas? `s01/receitas-s01.zip` do GitHub: descompactar, já traz o Git.

~16 minutos, até à pausa.

SOLUÇÃO

# Branches no site das receitas

RECEITAS

```
$ git switch -c nova-receita
$ git add receitas/arroz-doce.html
$ git commit -m "Acrescenta a página do arroz-doce"
$ git commit -am "Acrescenta o arroz-doce à lista"

$ git switch main
$ git switch -c cores
$ git commit -am "Muda a cor do cabeçalho"

$ git switch main
$ git merge nova-receita  # fast-forward
$ git merge cores  # merge commit
$ git branch -d nova-receita cores
$ git log --oneline --graph
```

## O que interessava

Cada coisa no seu branch; o merge feito a partir do main; os dois casos de merge.

## Os erros mais comuns

O commit no branch errado; o merge ao contrário; a página nova sem git add (o -am não a apanha).

PARTE 3

# Conflitos

quando dois branches mudam a mesma linha

**O QUE É → RESOLVER, JUNTOS → O EXERCÍCIO 4.2**

TEORIA

# O Git pergunta: qual das duas?

PREFACIO.TXT, DEPOIS DO MERGE

```
<<<<<<< HEAD
Este livro é um diário de bordo do Git.
=======
Este livro ensina Git a quem nunca o usou.
>>>>>>> prefacio-novo
```

`<<<<<<<` **HEAD** o que está no main (onde estou)
`=======` a separação
`>>>>>>>` **prefacio-novo** o que vem do branch

1. **git status**
   Diz quais ficheiros têm conflito: "both modified".

2. **Escolher no ficheiro**
   Ficar com uma, com a outra ou com as duas. E apagar os marcadores.

3. **git add ficheiro**
   Diz ao Git: "este já está resolvido".

4. **git commit**
   Fecha o merge (a mensagem já vem preparada).

**Desistir? git merge --abort**

Volta tudo a como estava antes do merge.

# Provocar e resolver um conflito

1. git switch -c prefacio-novo · mudar a 1.ª linha do prefacio.txt · commit -am "Reescreve o prefácio"
2. git switch main · mudar a MESMA linha de outra maneira · commit -am "Muda o prefácio"
3. git merge prefacio-novo: CONFLICT! · git status: both modified
4. No VS Code: os marcadores e os botões (Accept Current / Incoming / Both) · deixar o texto final
5. git add prefacio.txt · git commit (a mensagem já vem feita: gravar e fechar)
6. git log --oneline --graph --all · git branch -d prefacio-novo

RESUMO

# O que escrevemos: o conflito

LIVRO

```
$ git switch -c prefacio-novo
$ git commit -am "Reescreve o prefácio"
$ git switch main
$ git commit -am "Muda o prefácio"

$ git merge prefacio-novo
CONFLICT (content): Merge conflict in prefacio.txt

$ git status
  both modified:   prefacio.txt

# escolher, apagar os marcadores, gravar
$ git add prefacio.txt
$ git commit
$ git branch -d prefacio-novo
```

## Um conflito é uma pergunta

O Git mostra as duas versões e espera pela nossa escolha.

## Resolver = editar + add + commit

E nunca deixar os marcadores <<<<<<< ======= >>>>>>> no ficheiro.

EXERCÍCIO

# Um conflito nas receitas

1. Branch `titulo-verao`: mudar o `<h1>` do `index.html` para "Receitas de verão" e fazer o commit
2. No `main`: mudar o MESMO `<h1>` para outro título e fazer o commit
3. Juntar o `titulo-verao` ao `main`: o conflito
4. Resolver: ficar com um dos títulos (ou juntar os dois), sem marcadores, add e commit
5. Repetir com a cor do cabeçalho no CSS, mas desta vez desistir com `git merge --abort`
6. Colar o `git log --oneline --graph` no chat e apagar os branches

## Regras

Na dúvida: git status (diz sempre o que falta).

Abram o index.html no browser antes do commit: com marcadores, a página aparece estragada.

~13 minutos.

SOLUÇÃO

# Um conflito nas receitas

RECEITAS

```
$ git switch -c titulo-verao
$ git commit -am "Muda o título para Receitas de verão"
$ git switch main
$ git commit -am "Muda o título para As receitas da avó"

$ git merge titulo-verao
CONFLICT (content): Merge conflict in index.html
# escolher o <h1>, apagar os marcadores
$ git add index.html
$ git commit

# a cor: o conflito e desistir
$ git merge cores-novas
$ git merge --abort
$ git branch -D titulo-verao cores-novas
```

## O que interessava

Ler os marcadores, escolher e apagá-los; add e commit; e o --abort como saída de emergência.

## O -D

O branch das cores nunca foi juntado: o -d recusa apagá-lo. O -D força (perde-se esse trabalho).

PARTE 4

# .gitignore e voltar atrás

o que o Git não deve ver, e como desfazer commits

**.GITIGNORE → AMEND, REVERT, RESET → JUNTOS → BOAS PRÁTICAS**

TEORIA

# O que o Git não deve ver

.GITIGNORE (NA RAIZ DO PROJETO)

```
# o sistema operativo
.DS_Store
Thumbs.db

# segredos: nunca no Git!
.env

# o que se instala ou se gera
node_modules/
dist/

# rascunhos meus
notas-privadas.txt
*.log
```

### 1 O que ignorar

Lixo do sistema (.DS_Store), passwords e chaves (.env), o que se instala (node_modules) ou se gera (dist).

### 2 Já estava no Git?

O .gitignore só vale para ficheiros que o Git ainda não segue. Para deixar de seguir um: `git rm --cached ficheiro`.

### 3 Não começar do zero

Há modelos prontos por tecnologia: github.com/github/gitignore.

TEORIA

# Corrigir e desfazer commits

1

### amend: corrigir o último

git commit --amend -m "…" troca a mensagem do último commit (e junta-lhe o que estiver na caixa).

2

### revert: desfazer com segurança

git revert <hash> cria um commit novo que faz o contrário. O histórico fica todo: é o seguro.

3

### reset: apagar commits

git reset --hard HEAD~1 apaga o último commit e as mudanças. Não há volta: só em commits que ninguém viu.

**REVERT: O HISTÓRICO GUARDA O ERRO E A CORREÇÃO**

```
$ git revert HEAD
3c1f0aa (HEAD -> main) Revert "Apaga o capítulo 1"
8b2e7d4 Apaga o capítulo 1
```

### A regra de ouro

No que já foi partilhado (no GitHub, na 2.ª): só revert. O amend e o reset reescrevem o histórico.

# No livro: ignorar, corrigir, desfazer

1. Criar notas-privadas.txt · git status: o Git vê-o
2. Criar .gitignore com notas-privadas.txt e .DS_Store · git status: desapareceu
3. git add .gitignore · git commit -m "gitignore" (má mensagem!) · git commit --amend -m "Acrescenta o .gitignore"
4. "Por engano": git rm capitulos/capitulo1.txt · git commit -m "Apaga o capítulo 1"
5. git revert HEAD: o capítulo volta, num commit novo (gravar e fechar a mensagem)
6. git log --oneline: o erro e a correção, os dois no histórico

TEORIA

# Em equipa: nomes de branches e mensagens

1

## Um branch por tarefa

Pequeno, com vida curta: junta-se ao main e apaga-se.

2

## Nomes que dizem o quê

feat/nova-receita, fix/link-do-menu, docs/readme.

3

## O main funciona sempre

Nunca trabalhar diretamente no main da equipa.

**CONVENTIONAL COMMITS: TIPO: DESCRIÇÃO**

```
feat: acrescenta a receita do arroz-doce
fix: corrige o link do menu para as sobremesas
style: muda a cor do cabeçalho
docs: acrescenta o README
refactor: reorganiza as pastas das imagens
chore: acrescenta o .gitignore
```

## Para quê?

Lê-se o histórico de relance (o que é novo, o que é correção) e há ferramentas que geram as notas de cada versão a partir das mensagens. Em português ou em inglês: segue-se a regra da equipa.

EXERCÍCIO

# .gitignore e voltar atrás nas receitas

1. Um .gitignore nas receitas (.DS_Store, notas-privadas.txt, *.log) e o commit chore:
2. Um branch feat/… com uma receita nova, juntado ao main, com mensagens Conventional Commits
3. Apagar "por engano" uma página de receita, fazer o commit, e desfazê-lo com git revert
4. Um commit com uma má mensagem, corrigido com git commit --amend
5. Colar o git log --oneline --graph no Moodle (fórum do módulo)

## Para treinar

Learn Git Branching: os níveis "Introduction Sequence" (commit, branch, merge) e "Ramping Up" (HEAD, reset, revert).

learngitbranching.js.org

RESUMO DA SESSÃO

# O que levamos de hoje

1

### Branches

git switch -c nome para criar, git switch para mudar. Commit antes de mudar.

2

### Merge

A partir de quem recebe: git switch main e git merge nome. Fast-forward ou merge commit.

3

### Conflitos

Escolher no ficheiro, apagar os marcadores, add e commit. Ou git merge --abort.

4

### Ignorar e voltar atrás

.gitignore; --amend (o último), revert (seguro), reset (perigoso).

### O desenho

git log --oneline --graph --all: na dúvida, ver onde estão as etiquetas. E o git status, sempre.

ATÉ 2.ª FEIRA

# Próxima sessão: o GitHub

1

## Sessão 3 · 2.ª, 16 nov

O repositório na nuvem: push, pull e clone, trabalhar em equipa com Pull Requests e publicar um site.

2

## Tragam

Uma conta no GitHub (github.com), com o email que puseram no git config. E o livro e as receitas de hoje.

3

## Para casa

O 4.3 (.gitignore e voltar atrás nas receitas) e o Learn Git Branching.

Snippets, enunciados e soluções: github.com/JoaoGoncalves/flag-frontend-2026 · pasta 03-git-github

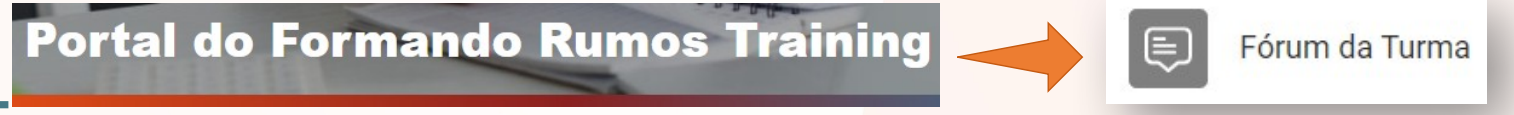
# Formador

Caro(a) formador(a),

Recordamos que no final de cada sessão têm de garantir os registos inerentes ao curso:

**Sumários** – Módulo do Formador 

**Presenças** - Módulo do Formador 

**Materiais** – Moodle 

FLAG

www.flag.pt

**LISBOA** Edifício Mirage, R. Dr. Eduardo Neves, Nº3, 1050-077 Lisboa email: querosabermais@flag.pt Tel. 213 560 606

**PORTO** Rua Oliveira Monteiro, Nº 168, 4050–438 Porto email: querosabermais@porto.flag.pt Tel. 226 094 789



*uma marca*
Rumos training | Knowledge sharing